

格点 QCD 强子关联函数自动生成引擎——核心物理算法与 代码-物理映射穷尽剖析

opencode pure

2026 年 8 月 15 日

摘要

本项目 LQCD Master 为 **代码-物理交叉向**项目：以自然语言物理任务为输入，经 Planner/Executor 双 Agent 管线产出 PyQUDA 可执行代码（依据：全库 74 个 Python 文件共 263050 行，核心为 utils/generate_einsum 收缩引擎与 core_architecture 编排层）。本报告穷尽枚举 11 个核心候选（C1-C11），其中 5 项深挖、4 项浅析、2 项排除。最深剖析结论：**wicklib 自动 Wick 收缩引擎 + 三夸克 576 拓扑预计算结构因子法**为最具独特竞争力部分——前者把场论级收缩（费米子符号、 γ 代数、色结构）编码为邻接矩阵与位掩码代数，自动产出 einsum 代码；后者把 576 个拓扑简化为运行时 576 次标量 MAC（约 μs 量级），替代约十万行生成代码（sink_pnn_full.py 达 100829 行）。

目录

1	项目定位与核心部分清单	2
1.1	项目性质判定	2
1.2	核心部分总清单（穷尽枚举）	2
2	核心部分深度剖析	3
2.1	C1: wicklib 自动 Wick 收缩引擎	3
2.1.1	物理推导链：两点关联函数的 Wick 收缩	3
2.1.2	算法原理：邻接矩阵与符号计数	4
2.1.3	γ 代数的位掩码实现	4
2.2	C2: 强子算符构造——介子/重子/双夸克块	5
2.3	C3/C10: 两点关联函数生成链与命名转换层	6
2.3.1	介子 2pt 格式化	7
2.4	C4: 三点关联函数——拓扑枚举与顺序源	8
2.5	C5: 三夸克 576 拓扑预计算——结构因子法	8
2.6	C6-C9: 浅析部分	9
3	独特性与竞争力评估	9
4	关键片段与公式	9

5 参考源清单

10

6 结论

11

1 项目定位与核心部分清单

1.1 项目性质判定

要点

项目性质: 代码-物理交叉向 (依据: 核心模块为物理计算代码 `utils/generate_einsum` (约 2 500 行手写引擎 + 生成产物 20 万行) + 物理知识库 `skills/ (lqcd-physics-correlator/spectrum、pyquda-gauge/tool` 等 5 个技能与参考文档) + Agent 编排层 `core_architecture` 与 `prompts/`; 代码与物理内容占比相当, 且存在紧密的逐符号对应关系)。

1.2 核心部分总清单 (穷尽枚举)

编号	核心部分	处理态	独特性概述
C1	wicklib 自动 Wick 收缩引擎	深挖	邻接矩阵表示收缩拓扑、费米子符号自动计入、 γ 代数位掩码化
C2	强子算符构造 (<code>hadron_operator</code> + <code>wicklib/operator</code>)	深挖	介子/重子/双夸克块对象模型, 转置与厄米共轭自动传播
C3	两点关联函数生成链 (<code>two_pt</code>)	深挖	收缩 $\rightarrow$ <code>einsum</code> $\rightarrow$ PyQUDA 五步管线, 含 γ_5 -厄米性折叠
C4	三点关联函数生成链 (<code>three_pt</code>)	深挖	拓扑枚举 + 顺序源 (<code>sequential source</code>) 代码生成
C5	三夸克 576 拓扑预计算 (<code>three_baryon_opt</code>)	深挖	结构因子离线预计算, 运行时标量 MAC 替代十万行代码
C6	大规模生成代码产物 (<code>sink_pnn_full</code> 等)	浅析	引擎输出 10 万行级生产代码 (产物, 非引擎本体)
C7	Agent 编排 (<code>core_architecture</code>)	浅析	Planner critique-rewrite + Executor 静态分析/动态测试
C8	物理技能知识库 (<code>skills/</code>)	浅析	强子算符/质量提取公式的注入式知识库

编号	核心部分	处理态	独特性概述
C9	静态检查与 Slurm 提交 (utils/static_check 等)	浅析	PyQUDA 代码静态分析与作业队列监控
C10	命名转换层 (__wick_translate)	深挖	wicklib 符号 $\leftrightarrow$ PyQUDA 变量名的双向映射
C11	experiments/ 与 benchmark/ 数据目录	排除	实验输出与基准数据 (非核心逻辑, 数据体量占比大)

本节给出穷尽枚举与三态分配: 6 深挖 (C1/C2/C3/C4/C5/C10)、3 浅析 (C6–C9)、2 排除 (C11 及 three_pt/_to_delete 废弃样例)。其中 C10 命名转换层与 C1–C4 均为生成链的必经环节且各有独立算法内容, 故计入深挖。以下各节按物理图像 $\rightarrow$ 算法 $\rightarrow$ 映射三层剖析。

2 核心部分深度剖析

2.1 C1: wicklib 自动 Wick 收缩引擎

物理图像

物理图像: 强子关联函数 $\langle \mathcal{O}_{\text{snk}}(t) \mathcal{O}_{\text{src}}^\dagger(0) \rangle$ 对路径积分求值等价于对费米场做**全配对** (Wick 收缩): 每个配对贡献一条传播子 $S(x, y)$, 配对的交叉数决定费米子符号 $(-1)^{N_{\text{cross}}}$ 。引擎把这一场论操作编码为”算符张量流 $\rightarrow$ 邻接矩阵 $\rightarrow$ einsum 字符串”三段流水。

2.1.1 物理推导链: 两点关联函数的 Wick 收缩

以 $\pi^+ = \bar{d} \gamma_5 u$ 为例 (物理知识库 skills/lqcd-physics-correlator/reference/pion_mass.md: 7-30):

$$C_\pi(p; t, 0) = \langle \bar{d}(x) \gamma_5 u(x) \bar{u}(y) \gamma_5 d(y) \rangle = - \sum_{x, y} e^{-ip \cdot (x-y)} \text{Tr} [\gamma_5 S_u(x, y) \gamma_5 S_d(y, x)], \quad (1)$$

依据: Wick 定理 (唯一配对, 负号来自费米子反对易); γ_5 -厄米性:

$$S(y, x) = \gamma_5 S^\dagger(x, y) \gamma_5, \quad S_{\text{bwd}} = S_{\text{fwd}}^\dagger \text{ (代码中折叠进 } \gamma \text{ 插入)}. \quad (2)$$

在 u/d 味简并下 ($S_u = S_d = S_l$):

$$C_\pi(p; t, 0) = \sum_{x, y} e^{-ip \cdot (x-y)} \text{Tr} [S_l^\dagger(x, y) S_l(x, y)], \quad (3)$$

对应代码输出 `contract('wtzyxCBba, wtzyxCBba -> t', prop_l.conj(), prop_l)` (utils/generate_einsum/MANUAL.md:102-107 确证)。**极限校验:** $m_q \rightarrow \infty$ 时 $S_l \rightarrow 0$, $C_\pi \rightarrow 0$ (对应), 满足退化要求。

2.1.2 算法原理：邻接矩阵与符号计数

核心算法

核心算法：Wick 配对穷举 + 邻接矩阵编码。输入为算符的张量流（QuarkFieldTensor 与 Spin-ColorTensor 序列），输出为（系数, einsum 下标串, 操作数表）。复杂度：配对数为 $\prod_f n_f!$ (n_f 为第 f 味夸克对数目)； γ 乘法为 $O(1)$ 位掩码运算。

```

1  def pair_quark_antiquark(
2      self, pairs: List[Tuple[int, QuarkFieldTensor, QuarkFieldTensor]], quark_fields: List[QuarkFieldTensor]
3  ) -> Generator[List[Tuple[int, QuarkFieldTensor, QuarkFieldTensor]], None, None]:
4      if not quark_fields:
5          yield list(pairs)
6          return
7      for quark_idx, quark in enumerate(quark_fields):
8          if not quark.antiquark:
9              break
10         else:
11             raise ValueError("unmatched_antiquark_found")
12         found = False
13         for antiquark_idx, antiquark in enumerate(quark_fields):
14             if antiquark.antiquark and quark.flavor == antiquark.flavor:
15                 found = True
16                 distance = quark_idx - antiquark_idx
17                 distance = -distance if distance < 0 else distance - 1
18                 pairs.append((-1) ** distance, quark, antiquark)
19                 yield from self.pair_quark_antiquark(
20                     pairs, [q for q in quark_fields if q is not quark and q is not antiquark]
21                 )
22                 pairs.pop()
23         if not found:
24             raise ValueError("unmatched_quark_found")

```

符号计入：第 78–80 行按配对距离奇偶给出 $(-1)^d$ ；**正确性依据：**同味配对穷举（第 75–84 行），未匹配夸克报错（第 73/86 行），保证每项可复核。物理对象：每个 $S_{\text{snk},\text{src}}$ 边（utils/generate_einsum/wicklib/adjacency.py:30–35）。

2.1.3 γ 代数的位掩码实现

$$\Gamma_i \Gamma_j = \text{sign}(i, j) \Gamma_{i \oplus j}, \quad \text{sign}(i, j) = (-1)^{|i \wedge j|} \text{按位奇偶修正}, \quad (4)$$

其中 $i, j \in \{0, \dots, 15\}$ 为 4 位掩码（utils/generate_einsum/wicklib/gamma.py:4–55 确证：第 45 行 XOR 求积、第 47–52 行分权重符号表 popsign）。**转置/厄米/狄拉克共轭**（第 57–78 行）均以 popcnt 奇偶给出系数， $O(1)$ 完成——这是引擎能快速化简任意 γ 链的基础。

代码-物理映射

映射原则：收缩引擎中每个对象均可双向追溯至物理量。

代码符号	物理对象	物理含义与参考源
QuarkFieldTensor	夸克场 $q_\alpha^a(x)$	味/位置/旋量指标/色指标
SpinGammaTensor	$\Gamma_{\alpha\beta}$	γ 矩阵旋量分量
ColorDeltaTensor	δ^{ab}	色单态收缩（介子）
ColorEpsilonTensor	ϵ^{abc}	全反对称色张量（重子）
AdjacencyEdge	传播子 $S(x, y)$	wicklib/adjacency.py:21-35
pair_quark_antiquark	Wick 配对	wicklib/correlator.py:63-86

2.2 C2: 强子算符构造——介子/重子/双夸克块

物理图像

物理图像: 介子算符 $M = \bar{q} \Gamma q'$ 是色单态双线性；重子算符 $B = \epsilon_{abc} (q_a^T C \Gamma' q_b) q_c$ 由双夸克（颜色反对称对）与旁观夸克组成， $C\gamma_5$ ($J^P = \frac{1}{2}^+$ 八重态) 与 $C\gamma_1$ ($J^P = \frac{3}{2}^+$ 十重态) 对应不同的味道-旋量对称性。

```

1 def baryon_operator(a_flavor: str, b_flavor: str, c_flavor: str,
2                     diquark_gamma: str = "Cg5",
3                     location: str = "") -> Operator:
4     """B(x)=\sum_{abc}\sum_{\alpha\beta\gamma}\sum_{\alpha'\beta'\gamma'}(diquark_gamma)_{\alpha\beta\gamma}\sum_{\alpha'\beta'\gamma'}q_{\alpha'}^{\alpha}q_{\beta'}^{\beta}q_{\gamma'}^{\gamma}
5
6     \sum_{\alpha\beta\gamma}\sum_{\alpha'\beta'\gamma'}The full diquark gamma is, e.g.:
7     \sum_{\alpha\beta\gamma}\sum_{\alpha'\beta'\gamma'}diquark_gamma="Cg5"\sum_{\alpha\beta\gamma}\sum_{\alpha'\beta'\gamma'}(J=1/2 octet, flavor-antisymmetric diquark)
8     \sum_{\alpha\beta\gamma}\sum_{\alpha'\beta'\gamma'}diquark_gamma="Cg1"\sum_{\alpha\beta\gamma}\sum_{\alpha'\beta'\gamma'}(J=3/2 decuplet, flavor-symmetric diquark)
9     """
10    1, 1, 1 = "A", "B", "C"
11    a1, b1, c1 = "a", "b", "c"
12    is_baryon = True
13    return Operator([
14        epsilon(a1, b1, c1),
15        quark(a_flavor, 1, a1),
16        gamma(diquark_gamma, 1, 1), # C @ _mat connects diquark
17        quark(b_flavor, 1, b1),
18        quark(c_flavor, 1, c1), # spectator (no )
19    ], is_baryon)

```

对应 wicklib 对象层: `QuarkBlock.to_tensor` 引入 δ^{ab} 、 $\Gamma_{\alpha\beta}$ 与新旋量/色指标 (`wicklib/operator.py`: 181-199 确证); `DiquarkBlock` 用 ϵ^{abc} 与转置夸克 (第 246-265 行) 编码 q_a^T ——转置在对象模型中作为标记 (transpose 标志) 传播, 避免显式矩阵转置, 这是“代码-物理——对应”的典型例证。

$C\gamma_5$ 选择依据: 色反对称 + 旋量反对称 ($C\gamma_5$ 的交换对称性) 强制味道反对称, 即八重态; $C\gamma_1$ 对应味道对称, 即十重态——代码注释 (`hadron_operator.py`:86-88) 直接记录该物理规则, 属“物理进代码”。

2.3 C3/C10: 两点关联函数生成链与命名转换层

核心算法

核心算法: 五步管线 (MANUAL.md:40-63): 算符定义 → 张量 → wicklib 块 (_wick_translate.py:52-76) → Wick 收缩 + 化简 (two_pt/contract.py:95-146) → einsum 提取 (wicklib/adjacency.py:157-214) → PyQUDA 格式化 (two_pt/codegen.py:27-117)。

```

1  def to_einsum(self) -> Tuple[Union[int, float, complex], str, List[str]]:
2      index_map = IndexMap()
3      subscripts: List[str] = []
4      operands: List[str] = []
5
6      SpinEinsumIndex.reset()
7      ColorEinsumIndex.reset()
8      for tensor in self.tensors:
9          if isinstance(tensor, SpinGammaTensor) and tensor.gamma.index == 0:
10             index_map.delta_spin(tensor.left, tensor.right, SpinEinsumIndex)
11          elif isinstance(tensor, SpinGammaTensor):
12             A = index_map.setdefault_spin(tensor.left, SpinEinsumIndex)
13             B = index_map.setdefault_spin(tensor.right, SpinEinsumIndex)
14             subscripts.append(A + B)
15             operands.append(f"gamma({tensor.gamma.index})")
16          elif isinstance(tensor, SpinProjectorTensor):
17             A = index_map.setdefault_spin(tensor.left, SpinEinsumIndex)
18             B = index_map.setdefault_spin(tensor.right, SpinEinsumIndex)
19             subscripts.append(A + B)
20             operands.append("projector")
21          elif isinstance(tensor, ColorDeltaTensor):
22             index_map.delta_color(tensor.left, tensor.right, ColorEinsumIndex)
23          elif isinstance(tensor, ColorEpsilonTensor):
24             a = index_map.setdefault_color(tensor.left, ColorEinsumIndex)
25             b = index_map.setdefault_color(tensor.middle, ColorEinsumIndex)
26             c = index_map.setdefault_color(tensor.right, ColorEinsumIndex)
27             subscripts.append(a + b + c)
28             operands.append("epsilon")
29
30      for row, row_vec in enumerate(self.matrix):
31          for col, edge in enumerate(row_vec):
32              if edge.flavor is not None:
33                  subscripts.append(
34                      "...
35                      + index_map.get_spin(f" {row}")
36                      + index_map.get_spin(f" {col}")
37                      + index_map.get_color(f"a{row}")
38                      + index_map.get_color(f"b{col}")
39                  )
40                  link_prefix = (
41                      f"W_{edge.prefix.origin}_{edge.prefix.location}" if isinstance(edge.prefix, QuarkLink) else "
42                      ↪ "
43                  )
44                  link_suffix = (
45                      f"_W_{edge.suffix.location}_{edge.suffix.origin}" if isinstance(edge.suffix, QuarkLink) else "

```

```

45         )
46         nabla_prefix = f"D_{edge.prefix.direction}_" if isinstance(edge.prefix, QuarkDerivative) else ""
47         nabla_suffix = f"_D_{edge.suffix.direction}" if isinstance(edge.suffix, QuarkDerivative) else ""
48         smearing_prefix = "S_" if isinstance(edge.prefix, QuarkSmearing) else ""
49         smearing_suffix = "_S" if isinstance(edge.suffix, QuarkSmearing) else ""
50         conj = ".conj()" if edge.conjugate else ""
51         operands.append(
52             f"{link_prefix}{nabla_prefix}{smearing_prefix}"
53             f"propag_{edge.flavor}_{edge.sink}_{edge.source}"
54             f"{smearing_suffix}{nabla_suffix}{link_suffix}"
55             f"{conj}"
56         )
57
58     return self.factor, ",".join(subscripts) + "->...", operands

```

要点: 第 165–184 行对 γ 矩阵与 ϵ 建立子标; 第 186–212 行把每个传播子边写为 `propag_f_sink_source` 形式的操作数 (flavor/sink/source 分别表示味、末态、初态位置); 第 196–211 行把链接、导数、涂抹前缀编码进变量名 (W 链变量、 D 协变导数、 S 涂抹——映射层 C10 的核心)。

反向传播子折叠 (`_wick_translate.py:93–122` 确证): $S_{\text{bwd}} = G5 @ S.dag() @ G5$ 由 `_rename_op` 自动生成, 即公式 (2) 在代码生成层的直接体现; 同一性化简 `_simplify_gammas` (第 125–151 行) 取消 $\gamma^2 = I$ 相邻对。

2.3.1 介子 2pt 格式化

`pyquda_format_contract` (`two_pt/codegen.py:47–117`) 识别 4 操作数介子型: γ_{snk} 插 $G5 @ \gamma$ 、 γ_{src} 插 $\gamma @ G5$ (第 74–75 行, 对应 `rho_mass.md` 参考约定), 并自动检测同味介子的断连图 (第 112–116 行, 标注 [DISCONNECTED], 需 all-to-all 方法——物理风险显式入码)。

代码-物理映射

映射表 (双向): 代码符号 $\leftrightarrow$ 物理对象。

代码符号	物理对象	物理含义与参考源
<code>prop_l / prop_s</code>	S_l/S_s	轻/奇异夸克传播子
<code>G5</code>	γ_5	<code>_wick_translate.py:84–86</code>
<code>Cg5 / Cg1</code>	$C\gamma_5/C\gamma_1$	双夸克, $J^P = 1/2^+/3/2^+$
<code>gtg5</code>	$\gamma_4\gamma_5$	非定域流
<code>Tmat</code>	$P_+ = (1 + \gamma_4)/2$	重子自旋投影
<code>wtzyx</code> 前缀	格点时空指标	t, z, y, x 张量维序
<code>.data</code>	格点场数组	PyQUDA LatticePropagator 存储

2.4 C4: 三点关联函数——拓扑枚举与顺序源

物理图像

物理图像: 三点函数 $\langle O_{\text{snk}} J O_{\text{src}}^\dagger \rangle$ 的收缩需先枚举”旁观夸克不变、两条流线在算符间交换”的拓扑（如 $p \rightarrow p$ 4 拓扑、 $\Omega^- \rightarrow \Omega^-$ 18 拓扑、全异味 1 拓扑，MANUAL.md:189-201 确证），再用**顺序源**（sequential source）方法以 $O(N)$ 次求逆实现（MANUAL.md:227-246: sink 块 $B \rightarrow$ 顺序传播子 $\rightarrow$ 最终 $\text{Tr}[G_{\text{seq}}^\dagger \Gamma_{\text{cur}} S_{\text{fwd}}]$ ）。

对应公式：

$$C_3(t_{\text{snk}}, t_{\text{cur}}) = \sum_{\vec{x}} \text{Tr} \left[\gamma_5 G_{\text{seq}}^\dagger \Gamma_{\text{cur}} S_{\text{fwd}} \right], \quad G_{\text{seq}}(t_{\text{snk}}) = [B \cdot S](t_{\text{snk}}). \quad (5)$$

极限校验: $t_{\text{cur}} \rightarrow t_{\text{snk}}$ 时三点退化为两点（断连顶角可忽略），与标准文献一致（推断，未实测）。拓扑枚举规则以味道匹配实现（three_pt/contract.py 与 MANUAL.md:378-391 确证）。

2.5 C5: 三夸克 576 拓扑预计算——结构因子法

核心算法

核心算法: pnL 三夸克两点函数 ($4u + 4d + 1s$ 共 $4! \times 4! \times 1! = 576$ 个 Wick 拓扑)。离线阶段用 wicklib + opt_einsum 预计算 576 个**结构因子** s_k （复常数，存为 np.array），运行时只需：

$$C_3(t) = \sum_{k=1}^{576} \sigma_k s_k \prod_{i=1}^9 \text{Tr}[S_i(t)], \quad (6)$$

σ_k 为费米子符号（预存数组）。复杂度： $O(576 \times 9)$ 次矩阵迹 + 576 次标量 MAC，约 μs （注释原话：“576 scalar MACs on CPU ($\sim \mu\text{s}$)”，three_baryon_opt/gen_pnL_fast.py:1-7 确证）。

```

1 # pnL 2pt: 576 precomputed structure factors
2 # Generated offline via wicklib + opt_einsum.
3 # No Tmat
4 #
5 # At run time: C3 = Σ sign_k × struct_k × Π Tr[propagator_i]
6 # All structure factors are precomputed as complex scalars.
7 # The run-time loop is 576 scalar MACs on CPU (~ s).
8
9 import cupy as cp, numpy as np
10 from pyquda import core

```

```

1 # Slot → propagator variable mapping
2 # slots 0-3: prop_l (u quarks)
3 # slots 4-7: prop_l (d quarks)
4 # slot 8: prop_s (s quark)
5
6 props = [prop_l.data] * 8 + [prop_s.data]
7
8 # Run-time loop

```



```

9 total = 0.0 + 0.0j
10 for k in range(576):
11     tr = 1.0
12     pairs = pairings[k]
13     for si in range(9):
14         tr *= complex(cp.trace(props[si]))
15     total += signs[k] * structs[k] * tr
16
17 C3_t = core.gatherLattice(cp.array([total]), [0])
18 if core.getMPIRank() == 0:
19     np.save('pnL_2pt.npy', cp.asnumpy(C3_t))

```

对比：朴素路径（three_baryon/gen_pnL_2pt.py:38-45）把 576 项写成 576 个 GPU einsum 调用（每项 24 个操作数、9 个传播子）；结构因子法把它压缩为 CPU 标量循环——代价是离线预计算（一次性成本）与传播子仅以迹 $\text{Tr}[S_i]$ 参与（要求 sink 端为点源构造，属应用范围限制）。**未验证：**本报告未在本机复跑 GPU 基准， μs 量级为文件注释原文，非本会话实测。

2.6 C6–C9: 浅析部分

- **C6 生成产物：**sink_pnn_full.py (100829 行) 与 sink_pnn_block.py (100806 行) 为 C5 引擎的朴素版输出，同为 576 拓扑的展开实现；两者规模是”自动生成 $\gg$ 手写”能力的最直观证据（**未验证：**仅按行数统计，未做逐行审阅）。
- **C7 Agent 编排：**orchestrator.py (816 行) 实现 Planner critique-rewrite 循环（core_architecture/orchestrator.py:109-133）与 Executor 生成 $\rightarrow$ 静态检查 $\rightarrow$ Slurm 测试提交 $\rightarrow$ 失败反馈重写的闭环（第 201–281 行），含队列状态机（squeue/sacct/scontrol 解析）。
- **C8 物理知识库：**skills/lqcd-physics-correlator 等 5 技能注入算符约定、 γ_5 -厄米性与质量提取公式（如 pion_mass.md 全文即公式 (3) 的推导笔记）。
- **C9 静态检查：**static_check.py (536 行) 对生成代码做 PyQUDA 特定静态分析，submit_tool.py 封装 sbatch/squeue。

3 独特性与竞争力评估

4 关键片段与公式

核心公式汇总：Wick 收缩 (1)、 γ_5 -厄米性 (2)、 γ 代数 (4)、三点顺序源 (5)、结构因子法 (6)。

关键代码片段（直引，各 ≤ 40 行）：Wick 配对 (C1 节)、 γ 乘法（wicklib/gamma.py:42-55）、einsum 提取 (C3 节)、结构因子运行时循环 (C5 节)。其中 γ 乘法片段如下：

```

1 def __matmul__(self, rhs: "Gamma") -> "Gamma":
2     if not isinstance(rhs, Gamma):
3         return NotImplemented
4     index = self.index ^ rhs.index
5     factor = self.factor * rhs.factor
6     if self.index & 0b1000:
7         factor *= Gamma.popsign[rhs.index & 0b0111]

```

独特点	对比基准	优势与证据
自动 Wick 收缩 + 费米子符号	手工推导（文献级）	576 拓扑无遗漏、符号自动计入 (<code>wicklib/correlator.py:63-86</code>)
γ 位掩码代数	sympy/显式 4×4 矩阵	每次乘法 $O(1)$ ，可穷举化简 (<code>wicklib/gamma.py:42-55</code>)
结构因子预计算	576 个 GPU einsum	运行时 576 标量 MAC 替代 10 万行代码 (<code>gen_pnL_fast.py:1-7</code>)
代码规模能力	手写 einsum	生成 10 万行级生产代码 (C6 实测行数)
Agent 闭环	单次生成	静态分析 + Slurm 动态测试 + 失败反馈重写 (C7)

表 4: 独特性评估（数据来源: 文件行数实测与代码结构分析）

```
8     if self.index & 0b0100:
9         factor *= Gamma.popsign[rhs.index & 0b0011]
10    if self.index & 0b0010:
11        factor *= Gamma.popsign[rhs.index & 0b0001]
12    # if self.index & 0b0001:
13    #     factor *= Gamma.popsign[rhs.index & 0b0000]
14    return Gamma(index, factor)
```

5 参考源清单

参考源	类型	内容/作用
<code>wicklib/correlator.py:63-86</code>	仓库	Wick 配对穷举与费米子符号
<code>wicklib/gamma.py:4-78</code>	仓库	γ 位掩码代数与共轭
<code>wicklib/adjacency.py:157-214</code>	仓库	einsum 字符串生成
<code>wicklib/operator.py:181-299</code>	仓库	夸克/双夸克块对象模型
<code>hadron_operator.py:59-99</code>	仓库	介子/重子算符定义
<code>_wick_translate.py:93-122</code>	仓库	γ_5 -厄米性与命名映射
<code>two_pt/codegen.py:27-117</code>	仓库	PyQUDA contract 格式化
<code>three_baryon_opt/gen_pnL_fast.py:1-7</code>	仓库	576 结构因子法声明
<code>MANUAL.md:40-63</code>	仓库	2pt 五步管线
<code>core_architecture/orchestrator.py:109-281</code>	仓库	Agent 编排闭环
<code>pion_mass.md:7-46</code>	仓库	π 两点函数推导笔记

6 结论

结论

穷尽剖析完成：核心候选 11 个（6 深挖 / 3 浅析 / 2 排除）。最强竞争力为 wicklib 自动 Wick 收缩引擎（C1）与三夸克结构因子预计算（C5）的 **组合**：前者把场论收缩机械化（邻接矩阵 + 位掩码 γ 代数 + 自动符号），后者把机械展开的产物再压缩为 576 个标量 MAC。整条链（算符 $\rightarrow$ Wick $\rightarrow$ einsum $\rightarrow$ PyQUDA 代码）形成”物理公式可直接执行”的闭环，辅以 Agent 编排层实现无人值守的物理计算流水线。

局限与风险

- C5 的 μs 性能为文件注释原文（`gen_pnL_fast.py:6`），本会话未在 GPU 上复测（本机无 PyQUDA/CUDA 环境，**未验证**）。
- 结构因子法适用范围受限：要求所有传播子以迹参与（点源/定点构造），非一般源情形不适用。
- 断连图（如 η_s 色环）仅标注 [DISCONNECTED] 并注释，未给出 all-to-all 实现（物理风险点）。
- 三态计数中 C6 生成产物 10 万行仅按行数判定，未逐行审阅正确性。
- 排除项 C11（`experiments/benchmark` 数据）与样例代码（`three_pt/_to_delete`）未纳入剖析，因其为数据/废弃代码。